UNIVERSIDAD DE LOS ANDES
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

LABORATORIO 3: ANÁLISIS CAPA DE TRANSPORTE Y SOCKETS

ISIS3204 – INFRAESTRUCTURA DE COMUNICACIONES

Profesora Yuri Pinto Rojas
Asist. De laboratorio: Nathalia Quiroga

Grupo 11
Camilo Puerto Peña - 202211362
Daniela Munar González - 202311392
Santiago Baquero Villa - 202220396

2026-10

Tabla de contenido:

1. Implementación del sistema TCP……
2. Implementación del sistema UDP……
3. Pruebas de comunicación (Publisher – Broker – Subscriber)……..
4. Capturas de tráfico en TCP (Wireshark)……..
5. Capturas de tráfico en UDP (Wireshark)……..
6. Comparación de desempeño de TCP y UDP……...…
7. Respuestas a las preguntas de análisis……..
8. URL…….
9. Bibliografía……...

1. Implementación del sistema TCP

Publisher:
El publisher TCP se implementó como el componente encargado de generar y enviar los mensajes relacionados con los eventos del partido al broker. Para esto, el programa crea un socket utilizando el protocolo TCP, lo que permite establecer una conexión confiable con el broker mediante la función connect(). Una vez establecida la conexión, el publisher solicita al usuario ingresar mensajes desde la consola, los cuales son enviados al broker utilizando la función send(). Este proceso se repite continuamente, permitiendo que el publisher transmita múltiples actualizaciones del partido en tiempo real hacia el broker, quien posteriormente se encarga de distribuirlas a los suscriptores conectados.

Broker:
El broker TCP se implementó como el encargado de recibir los mensajes enviados por los publishers y enviarlos a los demás clientes conectados. Para esto, el programa crea un socket usando TCP, el cual permite una comunicación confiable. Luego, se asocia a un puerto con la función bind() y se deja en espera de conexiones con listen(). Cuando un cliente se conecta, el broker lo acepta con accept() y guarda la conexión. Además, se usa la función select() para poder manejar varios clientes al mismo tiempo. Cuando un cliente envía un mensaje, el broker lo recibe con read() y lo reenvía a los otros clientes usando send(). De esta forma, el broker actúa como un intermediario que permite que los mensajes se distribuyan entre los clientes de manera correcta.

Subscriber:
El subscriber TCP se implementó como el componente encargado de recibir las actualizaciones del partido que son enviadas por los publishers y redistribuidas por el broker. Para esto, el programa crea un socket utilizando el protocolo TCP mediante la función socket(), lo que permite establecer una comunicación confiable con el broker. Posteriormente, el subscriber se conecta al broker utilizando la función connect(), indicando la dirección IP y el puerto en el cual el broker se encuentra escuchando conexiones.
Una vez establecida la conexión, el subscriber permanece a la espera de mensajes provenientes del broker. Para recibir estos mensajes se utiliza la función recv(), la cual permite leer los datos enviados a través del socket. Cada vez que el broker recibe un mensaje generado por algún publisher, este lo reenvía a los demás clientes conectados, incluyendo a los subscribers. Cuando el subscriber recibe uno de estos mensajes, el programa lo muestra en pantalla para que el usuario pueda visualizar las actualizaciones del partido en tiempo real.
Este proceso se ejecuta dentro de un ciclo continuo, lo que permite que el subscriber permanezca activo y reciba múltiples mensajes a medida que los publishers reportan nuevos eventos del partido. Gracias al uso del protocolo TCP, se garantiza que los mensajes lleguen completos, en el mismo orden en que fueron enviados y sin pérdida de información durante la transmisión.

2. Implementación del sistema UDP

Publisher:
El publisher UDP se implementó como el encargado de generar y enviar mensajes al broker utilizando el protocolo UDP. A diferencia de TCP, en este caso no se establece una conexión previa entre el publisher y el broker. El programa crea un socket UDP y envía cada mensaje de manera independiente mediante la función sendto(), indicando la dirección IP y el puerto del broker. Los mensajes se ingresan desde la consola y se transmiten como datagramas. Debido a las características de UDP, no se garantiza la entrega ni el orden de los mensajes, por lo que durante las pruebas es posible que algunos mensajes se pierdan o lleguen en un orden diferente.

Broker:
El broker UDP se implementó como el encargado de recibir mensajes y reenviarlos a los demás clientes, pero a diferencia de TCP, este no establece conexiones. En lugar de eso, utiliza datagramas, por lo que cada mensaje se envía de manera independiente usando las funciones recvfrom() y sendto().

Para poder reenviar los mensajes, el broker identifica a los clientes a partir de su dirección IP y puerto, guardándolos en una lista cada vez que recibe un mensaje nuevo. Debido a que UDP no garantiza la entrega ni el orden de los mensajes, es posible que algunos se pierdan o lleguen desordenados durante las pruebas.

Subscriber:

El subscriber UDP se implementó como el componente encargado de recibir los mensajes reenviados por el broker utilizando el protocolo UDP. A diferencia del caso TCP, en este protocolo no se establece una conexión previa entre el subscriber y el broker. En su lugar, el programa crea un socket utilizando socket() con el tipo SOCK_DGRAM, el cual permite enviar y recibir datagramas de manera independiente.
Para que el broker pueda identificar al subscriber como un cliente activo, el programa envía inicialmente un pequeño mensaje al broker utilizando la función sendto(). De esta forma, el broker registra la dirección IP y el puerto del subscriber dentro de su lista de clientes, permitiendo que posteriormente pueda reenviarle los mensajes que reciba.
Después de este registro inicial, el subscriber entra en un ciclo continuo donde espera la llegada de mensajes utilizando la función recvfrom(). Cada vez que el broker recibe un mensaje enviado por algún cliente, lo redistribuye a todas las direcciones registradas, incluyendo a los subscribers. Cuando el subscriber recibe uno de estos datagramas, el contenido del mensaje es mostrado en pantalla para que el usuario pueda observar las actualizaciones enviadas por los publishers.

Debido a las características propias del protocolo UDP, no se garantiza que todos los mensajes lleguen correctamente al subscriber ni que lo hagan en el mismo orden en que fueron enviados. Durante las pruebas es posible observar pérdida de mensajes o desorden en su recepción, lo cual permite comparar el comportamiento de UDP frente a TCP en términos de confiabilidad y control de transmisión.

3. Pruebas de comunicación (Publisher – Broker – Subscriber)

TCP:

Para realizar la prueba en TCP, primero se ejecutó el broker, el cual quedó escuchando en el puerto 8080, esperando que otros programas se conectaran. En la terminal se pudo observar que el broker se inicializó correctamente.

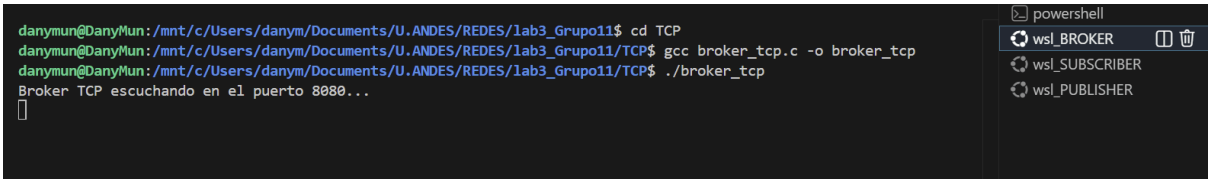


Figura 1: Broker TCP inicializandose y esperando conexiones.

Luego, se ejecutó el subscriber, el cual se conectó al broker sin problemas y quedó a la espera de mensajes. Esto indica que la conexión TCP se estableció correctamente entre el cliente y el servidor.

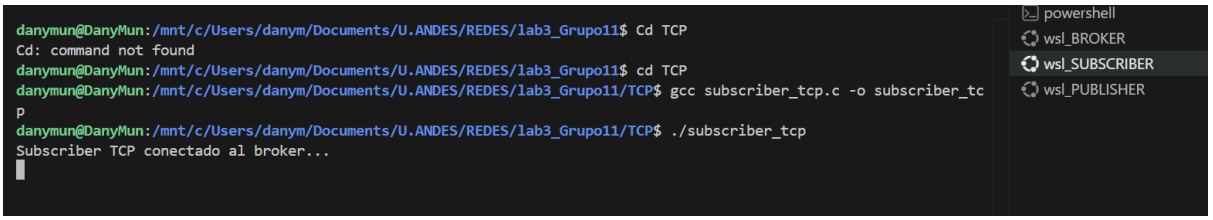


Figura 2: Subscriber TCP conectado al broker.

Después, se ejecutó el publisher, desde donde se enviaron mensajes de prueba simulando eventos, como por ejemplo un gol en un partido. Estos mensajes fueron enviados hacia el broker.

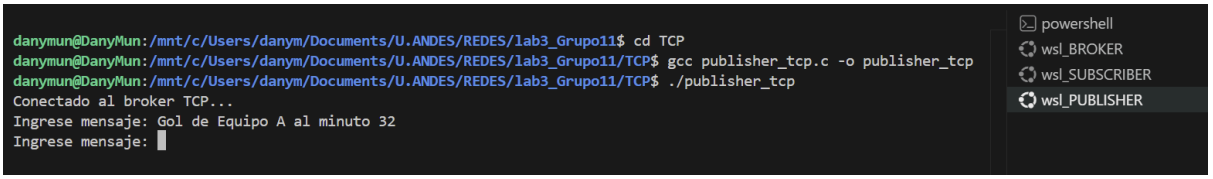


Figura 3: Publisher TCP enviando mensajes.

Se observó que el broker recibió correctamente los mensajes enviados por el publisher y los mostró en su terminal.

```
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ cd TCP
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ gcc broker_tcp.c -o broker_tcp
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ ./broker_tcp
Broker TCP escuchando en el puerto 8080...
Nuevo cliente conectado
Nuevo cliente conectado
Mensaje recibido: Gol de Equipo A al minuto 32
```

Figura 4: Broker TCP recibiendo mensajes del publisher.

Finalmente, el subscriber recibió los mismos mensajes que fueron enviados por el publisher, sin errores y en el mismo orden.

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ Cd TCP
Cd: command not found
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ cd TCP
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ gcc subscriber_tcp.c -o subscriber_tc
p
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ ./subscriber_tcp
Subscriber TCP conectado al broker...
Actualizacion recibida: Gol de Equipo A al minuto 32
```

Figura 5: Subscriber TCP recibiendo mensajes correctamente.

Con esta prueba se comprobó que TCP permite una comunicación confiable, ya que los mensajes llegan completos y en orden.

UDP:

Para la prueba en UDP, primero se ejecutó el broker UDP, el cual quedó escuchando en el mismo puerto 8080, pero sin establecer conexiones, ya que UDP no utiliza conexión directa entre los programas.

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

Nuevo cliente conectado
Mensaje recibido: Gol de Equipo A al minuto 32
Cliente desconectado
^C
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ CD..
CD..: command not found
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ cd ..
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ cd UDP
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ gcc broker_udp.c -o broker_udp
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ ./broker_udp
Broker UDP escuchando en el puerto 8080...
```

Figura 6: Broker UDP inicializándose.

Luego, se ejecutó el subscriber UDP. En este caso, el subscriber envió un mensaje inicial para poder ser registrado por el broker y así poder recibir mensajes posteriormente.

```
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ ./subscriber_tcp
Subscriber TCP conectado al broker...
Actualizacion recibida: Gol de Equipo A al minuto 32
^C
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ cd ..
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ cd UDP
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ gcc subscriber_udp.c -o subscriber
_udp
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ ./subscriber_udp
Subscriber UDP iniciado...
Actualizacion recibida: subscriber conectado
```

powershell
wsl_BROKER
wsl_SUBSCRIBER
wsl_PUBLISHER

Figura 7: Subscriber UDP listo y registrado en el broker.

Después, se ejecutó el publisher UDP, desde el cual se enviaron varios mensajes de prueba.

```
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ ./publisher_tcp
Conectado al broker TCP...
Ingrese mensaje: Gol de Equipo A al minuto 32
Ingrese mensaje: ^C
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ cd ..
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ cd UDP
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ gcc publisher_udp.c -o publisher_u
dp
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ ./publisher_udp
Publisher UDP listo para enviar mensajes
Ingrese mensaje: Gol del Equipo A al minuto 32
Ingrese mensaje:
```

powershell
wsl_BROKER
wsl_SUBSCRIBER
wsl_PUBLISHER

Figura 8: Publisher UDP enviando mensajes.

El broker recibió los mensajes enviados por el publisher y los mostró en pantalla.

```
CD..: command not found
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ cd ..
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ cd UDP
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ gcc broker_udp.c -o broker_udp
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ ./broker_udp
Broker UDP escuchando en el puerto 8080...
Mensaje recibido: subscriber conectado
Nuevo cliente UDP registrado
Mensaje recibido: Gol del Equipo A al minuto 32

Nuevo cliente UDP registrado
```

powershell
wsl_BROKER
wsl_SUBSCRIBER
wsl_PUBLISHER

Figura 9: Broker UDP recibiendo mensajes.

Finalmente, el subscriber UDP recibió los mensajes reenviados por el broker. Sin embargo, se observó que en algunos casos los mensajes pueden perderse o no llegar en el mismo orden.

```
Actualizacion recibida: Gol de Equipo A al minuto 32
^C
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/TCP$ cd ..
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11$ cd UDP
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ gcc subscriber_udp.c -o subscriber
_udp
danymun@DanyMun:/mnt/c/Users/danym/Documents/U.ANDES/REDES/lab3_Grupo11/UDP$ ./subscriber_udp
Subscriber UDP iniciado...
Actualizacion recibida: subscriber conectado
Actualizacion recibida: Gol del Equipo A al minuto 32
```

powershell
wsl_BROKER
wsl_SUBSCRIBER
wsl_PUBLISHER

Figura 10: Subscriber UDP recibiendo mensajes.

Con esta prueba se comprobó que UDP permite enviar mensajes de forma más simple, pero no garantiza que todos los mensajes lleguen correctamente ni en orden.

4. Capturas de tráfico en TCP (Wireshark)

Para la captura de tráfico TCP se utilizó Wireshark ejecutado en el entorno WSL, seleccionando la interfaz de loopback (lo) para poder observar el tráfico local generado por la aplicación. Antes de iniciar la ejecución de los programas, se comenzó la captura de paquetes con el fin de registrar todo el proceso desde el establecimiento de la conexión hasta el envío de los mensajes. Posteriormente, se aplicó el filtro tcp.port == 8080, lo cual permitió visualizar únicamente los paquetes relacionados con la comunicación entre el publisher, el broker y el subscriber.

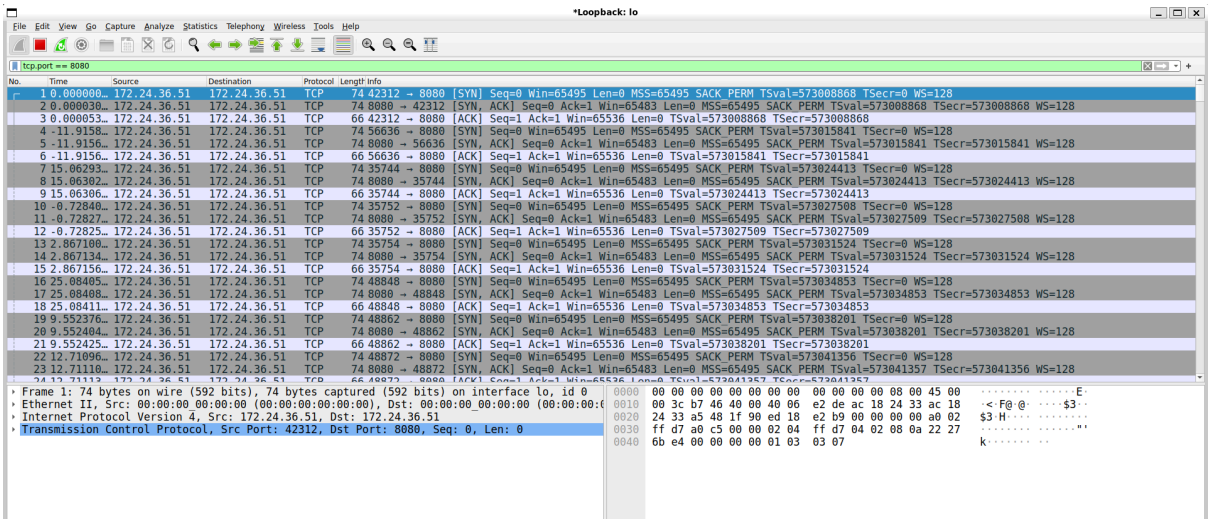


Figura 11: Aplicación del filtro tcp.port == 8080 en Wireshark.

Una vez iniciada la captura, se ejecutaron los diferentes componentes del sistema. Primero se levantó el broker, luego el subscriber y finalmente múltiples publishers para simular el envío concurrente de mensajes. Para esto se utilizaron varias terminales abiertas al mismo tiempo, evidenciando cómo múltiples clientes pueden conectarse simultáneamente al servidor. En estas terminales se pudo observar el envío de mensajes por parte de los publishers y la correcta recepción de estos en el subscriber.

Figura 12: Ejecución de múltiples terminales con broker, subscriber y publishers.

Durante la captura de tráfico, se pudo observar el proceso de establecimiento de conexión TCP, conocido como three-way handshake. Este proceso se identifica por la presencia de los paquetes SYN, SYN-ACK y ACK, los cuales permiten establecer una conexión confiable entre cliente y servidor antes de iniciar la transmisión de datos. Este comportamiento confirma que el protocolo TCP asegura una comunicación orientada a conexión.

Posteriormente, se identificó el envío de datos a través de paquetes marcados con las banderas PSH y ACK, los cuales contienen los mensajes enviados por los publishers. Estos paquetes representan la transmisión efectiva de la información a través de la red, y se pudo evidenciar que los mensajes fueron enviados correctamente hacia el broker y posteriormente reenviados al subscriber.

Adicionalmente, se utilizó la herramienta "Follow TCP Stream" de Wireshark para visualizar el flujo completo de la comunicación. Esta funcionalidad permitió observar de forma clara y organizada todos los mensajes intercambiados entre los clientes y el servidor, incluyendo los eventos enviados por los publishers. En esta vista se confirmó que los datos llegan completos y en el orden correcto.

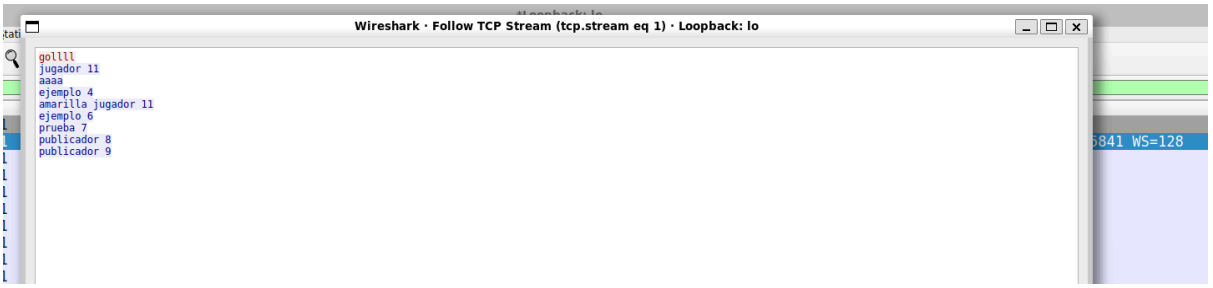


Figura 13: Visualización del flujo completo de comunicación TCP.

En general, a partir de las capturas realizadas, se pudo comprobar que el protocolo TCP garantiza una comunicación confiable, ya que establece una conexión previa, asegura la entrega de los datos y mantiene el orden de los mensajes. Además, al trabajar con múltiples publishers, se evidenció la capacidad del sistema para manejar varias conexiones simultáneas sin pérdida de información, lo cual es una de las principales ventajas de este protocolo.

5. Respuestas a las preguntas de análisis

1. ¿Qué ocurriría si en lugar de dos publicadores hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del broker bajo TCP y bajo UDP?

Si en lugar de dos publicadores hubiera cien enviando mensajes al mismo tiempo, el broker tendría que procesar y redistribuir una cantidad mucho mayor de mensajes. En TCP esto impactaría más el desempeño porque el broker debe mantener una conexión activa con cada

cliente, lo que implica manejo de buffers, control de flujo y confirmaciones de entrega. Esto aumenta el uso de CPU y memoria a medida que crece el número de conexiones.

En UDP el impacto sería menor porque no existen conexiones persistentes. El broker simplemente recibe datagramas y los reenvía a las direcciones registradas. Esto hace que UDP sea más liviano y escalable en escenarios con muchos emisores, aunque a cambio se pierde confiabilidad en la entrega de los mensajes.

2. Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario?

Si un suscriptor no recibe un mensaje en UDP, simplemente pierde ese evento. En una aplicación real esto podría significar que el usuario nunca se entere de que hubo un gol, lo que genera información incompleta o inconsistente sobre el partido.

TCP maneja mejor este escenario porque garantiza la entrega de los datos. Si un paquete se pierde durante la transmisión, TCP lo detecta y lo retransmite automáticamente. Además, asegura que los mensajes lleguen completos y en el mismo orden en que fueron enviados, lo que evita que se pierdan eventos importantes como goles o cambios de marcador.

3. En un escenario de seguimiento en vivo de partidos, ¿qué protocolo resultaría más adecuado?

Para un sistema de seguimiento en vivo de partidos, TCP resulta más adecuado porque garantiza la entrega correcta de todos los eventos importantes del juego. En este tipo de aplicaciones es más importante que la información llegue completa y en orden que tener una latencia mínima.

Aunque UDP es más rápido y tiene menos overhead, la posible pérdida de mensajes puede generar inconsistencias en la información que recibe el usuario. Por ejemplo, perder un mensaje de gol o de tarjeta podría afectar la comprensión del estado del partido. Por esta razón, para actualizaciones de marcador y eventos importantes, TCP suele ser la opción más confiable.

4. Compare el overhead observado en Wireshark entre TCP y UDP. ¿Cuál introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia?

En las capturas de Wireshark se observa que TCP introduce mayor overhead que UDP. Esto se debe a que el encabezado de TCP es más grande y además incluye mecanismos adicionales como números de secuencia, confirmaciones (ACK), control de flujo y control de congestión.

UDP, en cambio, tiene un encabezado mucho más simple y pequeño. Esto hace que cada mensaje tenga menos información adicional y, por lo tanto, el envío sea más eficiente en términos de tamaño de paquete y procesamiento. Sin embargo, esta eficiencia se logra

sacrificando confiabilidad, ya que UDP no verifica si los mensajes llegan correctamente al destino.

5. Si el marcador llega desordenado en UDP (por ejemplo, primero se recibe 2–1 y luego 1–1), ¿qué efectos tendría y cómo se podría solucionar?

Si los mensajes llegan desordenados, el usuario podría ver información incorrecta o confusa sobre el estado del partido. Por ejemplo, recibir primero un marcador de 2–1 y luego uno de 1–1 podría hacer parecer que el marcador retrocedió, lo cual no tiene sentido en un evento deportivo.

Una forma de solucionar este problema a nivel de aplicación sería incluir información adicional en los mensajes, como un número de secuencia o una marca de tiempo. De esta manera, el subscriber podría ordenar los eventos antes de mostrarlos al usuario o ignorar mensajes que correspondan a estados anteriores del partido. Esto permite compensar, en cierta medida, la falta de control de orden que tiene UDP.

6. ¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto?

Cuando aumenta el número de suscriptores, el broker tiene que reenviar cada mensaje a más clientes, por lo que el número de envíos crece proporcionalmente. En TCP esto afecta más el desempeño porque el broker mantiene una conexión activa con cada suscriptor, lo que implica más uso de memoria y mayor carga en la gestión de sockets. En UDP el impacto es menor, ya que el broker simplemente envía datagramas a las direcciones registradas sin mantener estado de conexión. Sin embargo, aunque UDP escala mejor en número de clientes, también aumenta la probabilidad de pérdida de mensajes cuando el tráfico es alto

7. ¿Qué sucede si el broker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión?

Si el broker se detiene inesperadamente, los clientes dejan de recibir mensajes inmediatamente. En TCP las conexiones se cierran o quedan inválidas, por lo que los clientes deben volver a conectarse cuando el broker esté disponible nuevamente. En UDP no existe una conexión persistente, por lo que los clientes simplemente dejan de recibir datagramas hasta que el broker vuelva a estar activo. Sin embargo, en ambos casos el sistema no recupera automáticamente la sesión y los clientes tendrían que volver a registrarse o reconectarse para seguir recibiendo mensajes.

8. ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué?

Garantizar que todos los suscriptores reciban exactamente al mismo tiempo una actualización es difícil debido a las diferencias de latencia en la red. Sin embargo, se puede mejorar la

sincronización enviando el mensaje inmediatamente a todos los clientes y agregando marcas de tiempo o números de evento en los mensajes para mantener consistencia. TCP facilita mejor este proceso porque asegura la entrega ordenada y confiable de los datos, evitando pérdidas o desorden en los mensajes. Esto permite que todos los suscriptores reciban la información de forma consistente, aunque el tiempo exacto de llegada pueda variar ligeramente.

9. Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró?

El broker utiliza más CPU y memoria cuando maneja múltiples conexiones TCP, ya que debe mantener una conexión activa con cada cliente y gestionar estructuras de control asociadas a cada socket. Además, TCP implementa mecanismos de control de flujo, confirmación de recepción y retransmisión de paquetes, lo que incrementa el procesamiento necesario. En UDP el uso de recursos es menor porque no se mantienen conexiones ni estados entre el broker y los clientes. El broker simplemente recibe datagramas y los reenvía, lo que reduce la carga de procesamiento y el consumo de memoria.

10. Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una combinación de ambos? Justifique con base en lo observado en el laboratorio.

Para un sistema real probablemente utilizaría una combinación de ambos protocolos. UDP sería útil para distribuir rápidamente las actualizaciones a un gran número de usuarios debido a su bajo overhead y mejor escalabilidad. Sin embargo, para eventos críticos o información que no debe perderse, como cambios de marcador o eventos importantes del partido, TCP sería más adecuado porque garantiza la entrega correcta y ordenada de los mensajes. De esta forma se aprovechan las ventajas de ambos protocolos: la eficiencia y rapidez de UDP junto con la confiabilidad de TCP.

6. Comparación de desempeño de TCP y UDP

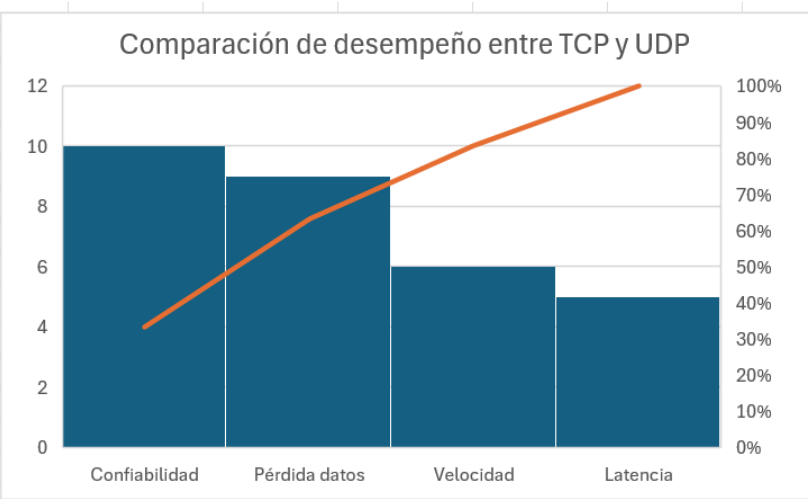
Para analizar el desempeño de los protocolos TCP y UDP, se realizó una comparación basada en características observadas durante la ejecución de las pruebas y el comportamiento esperado de cada protocolo. Aunque las pruebas prácticas se enfocaron principalmente en TCP, es posible contrastar su funcionamiento con UDP a partir de sus propiedades fundamentales y del comportamiento típico en redes.

En primer lugar, se observó que TCP establece una conexión antes de enviar datos, lo cual implica un proceso adicional conocido como three-way handshake. Este proceso introduce un pequeño retraso inicial, pero garantiza que la comunicación sea confiable. Por otro lado, UDP no requiere establecer una conexión previa, lo que permite un envío de datos más rápido, pero sin garantías de entrega.

En cuanto a la confiabilidad, TCP asegura que todos los mensajes lleguen completos y en el orden correcto, ya que cuenta con mecanismos de control de errores y confirmación de recepción. En contraste, UDP no garantiza la entrega de los mensajes ni el orden en que llegan, por lo que pueden presentarse pérdidas de información en escenarios de alta carga o congestión.

Respecto al manejo de múltiples mensajes, TCP mostró un comportamiento estable incluso cuando varios publishers enviaban datos simultáneamente, manteniendo la integridad de la información. En el caso de UDP, aunque permite una transmisión más rápida, es más susceptible a la pérdida de paquetes cuando aumenta la cantidad de mensajes enviados.

En términos de latencia, UDP presenta menor tiempo de respuesta debido a la ausencia de establecimiento de conexión y control de errores, mientras que TCP introduce mayor latencia por sus mecanismos de verificación, aunque esto se traduce en una comunicación más segura.



La gráfica muestra una comparación general entre TCP y UDP en términos de velocidad, confiabilidad, latencia y pérdida de datos. Los valores utilizados no corresponden a mediciones exactas obtenidas experimentalmente, sino a una **escala relativa cualitativa** construida a partir del comportamiento observado durante las pruebas realizadas en el laboratorio y de las características teóricas de cada protocolo. En este sentido, se asignaron valores más altos a UDP en velocidad y latencia debido a que no requiere establecimiento de conexión ni mecanismos de confirmación, lo que lo hace más rápido. Por otro lado, TCP obtuvo valores más altos en confiabilidad y control de pérdida de datos, ya que garantiza la entrega de la información y mantiene el orden de los mensajes mediante mecanismos de control. De esta manera, la gráfica permite visualizar de forma comparativa las principales diferencias de desempeño entre ambos protocolos.

7. URL

https://github.com/dmunarg2023/Redes_Lab3.git